Phase 4: Deployment — Complete Learning Guide

Table of Contents

---

1. The Big Picture

WHAT PHASE 4 ADDS

Phases 1-3 are Python scripts you run from the terminal.
Phase 4 turns them into SERVICES anyone can use:

```
Before Phase 4:
  python -m src.genai.rag_pipeline
  → prints answer in terminal
  → only the developer can use it

After Phase 4:
  Browser → http://localhost:8501
  → visual dashboard with charts, search, filters
  → ANY manager can use it

  curl http://localhost:8000/query -d '{"question":"stockout risk?"}'
  → JSON response
  → ANY system/app can call it
```

---

2. Key Concepts (from zero)

WHAT IS AN API?

API = Application Programming Interface.

Think of a restaurant menu. The menu is the API. It tells you:

- What you can order (endpoints)
- What information to give (request)
- What you'll get back (response)

You don't need to know HOW the kitchen works. You just need the menu.

Our API "menu":

- GET /health → "Is the server alive?" → {"status": "healthy"}
- GET /parts → "List all parts" → [list of parts with metadata]
- POST /query → "Ask a question" + {"question":"..."} → {"answer":"..."}
- GET /metrics → "Model accuracy" → {"mae": 2.77, "mape": 43.6}
- POST /forecast → "Get predictions" → [list of predictions]

WHAT IS REST?

REST = Representational State Transfer. Rules for API design:

- Use HTTP verbs: GET (read), POST (create/send), PUT (update), DELETE (remove)
- Each endpoint is a URL that represents a RESOURCE (/parts, /metrics)
- Responses are JSON (structured text that any language can read)
- Stateless: each request contains all info needed (no memory between requests)

WHAT IS FastAPI?

A Python framework for building REST APIs. You write a Python function,
add a decorator (@app.get), and FastAPI turns it into an HTTP endpoint.

```python
@app.get("/health")
def health_check():
    return {"status": "healthy"}
```

That's it. FastAPI automatically:

- Creates the HTTP endpoint at /health

- Converts the return dict to JSON
- Generates API documentation at /docs
- Validates request/response data
- Handles errors with proper HTTP status codes

WHAT IS STREAMLIT?

A Python framework for building dashboards. You write Python, Streamlit turns it into a web page.

```python
st.title("My Dashboard")
st.metric("Users", 42)
name = st.text_input("Your name:")
st.write(f"Hello {name}")
```

That's it. No HTML, no CSS, no JavaScript. Streamlit handles all the frontend rendering automatically.

WHAT IS DOCKER?

Docker packages your entire application (code + Python + libraries + models + data) into a CONTAINER that runs identically anywhere.

Without Docker:

- "It works on my laptop" but crashes on the server
- Different Python versions, missing libraries, wrong paths

With Docker:

- Build once, run anywhere
- Same Python, same libraries, same everything
- Deployment becomes: docker run supply-chain-orchestrator

---

3. File 1: main.py (The API Server)

WHAT IT DOES

Creates 5 REST API endpoints that expose all our Phase 1-3 work:

```
GET  /         → API info and available endpoints
GET  /health   → Server status + component health
GET  /parts    → List all parts with risk levels
POST /query    → Ask natural language questions (RAG)
GET  /metrics  → Model comparison (XGBoost vs ARIMA)
POST /forecast → Get demand predictions
```

KEY CODE PATTERNS

PYDANTIC MODELS for request validation:

python

```python
class QueryRequest(BaseModel):
    question: str = Field(..., description="Natural language question")
    top_k: int = Field(default=3, ge=1, le=10)
```

If someone sends {"question": 123} (number instead of string),
FastAPI automatically returns a 422 error: "question must be string."

LAZY LOADING for heavy components:

python

```python
_rag_pipeline = None

def get_rag_pipeline():
    global _rag_pipeline
    if _rag_pipeline is None:
        _rag_pipeline = create_rag_pipeline()
    return _rag_pipeline
```

The RAG pipeline loads only on the FIRST query, not at server start.
This makes the server start in <1 second instead of 30 seconds.

CORS MIDDLEWARE:

python

```python
app.add_middleware(CORSMiddleware, allow_origins=["*"])
```

This allows the Streamlit dashboard (running on port 8501) to call the API (running on port 8000). Without CORS, the browser blocks cross-origin requests for security.

HOW TO RUN

```bash
uvicorn src.api.main:app --reload
```

Then open: http://localhost:8000/docs

---

4. File 2: streamlit_app.py (The Dashboard)

WHAT IT DOES

Creates a 4-page web dashboard:

Page 1 — DASHBOARD: KPI cards, risk overview table, forecast chart
Page 2 — ASK A QUESTION: Text input + sample buttons + RAG answers
Page 3 — PARTS EXPLORER: Filter by demand type, expand part profiles
Page 4 — MODEL METRICS: Comparison table, bar charts, feature importance

KEY CODE PATTERNS

CACHING with @st.cache_data:

```python
@st.cache_data
def load_knowledge_base():
    with open("data/processed/knowledge_base.json") as f:
        return json.load(f)
```

Streamlit reruns the entire script on every interaction (button click, text input, slider move). @st.cache_data prevents reloading data every time — it loads once and caches the result.

SESSION STATE for persistent objects:

```python
if "rag_pipeline" not in st.session_state:
    st.session_state["rag_pipeline"] = create_rag_pipeline()
```

st.session_state persists across reruns. Without it, the RAG pipeline would be recreated on every button click (slow!).

HOW TO RUN

```bash
streamlit run streamlit_app.py
```

Then open: http://localhost:8501

---

5. Files 3-4: Dockerfile + docker-compose.yml

DOCKERFILE EXPLAINED

```dockerfile
FROM python:3.11-slim          # Start from Python base image
WORKDIR /app                    # Set working directory
COPY requirements.txt .        # Copy dependencies list
RUN pip install -r requirements.txt  # Install dependencies
COPY . .                       # Copy all project files
EXPOSE 8000 8501               # Declare which ports we use
CMD uvicorn ... & streamlit ... # Start both services
```

Each line creates a LAYER. Docker caches layers — if requirements.txt hasn't changed, Docker skips the pip install (saves minutes).

DOCKER-COMPOSE EXPLAINED

Runs two separate containers that talk to each other:

- api container → FastAPI on port 8000
- dashboard container → Streamlit on port 8501

The dashboard depends_on the api, so Docker starts the API first.

HOW TO RUN

```bash
```

```
# Build and start everything
docker-compose up --build

# Or without Docker Compose (single container)
docker build -t supply-chain-orchestrator .
docker run -p 8000:8000 -p 8501:8501 supply-chain-orchestrator
```

---

6. How to Execute Everything

WITHOUT DOCKER (for development)

bash

```
# Terminal 1: Start the API
uvicorn src.api.main:app --reload

# Terminal 2: Start the dashboard
streamlit run streamlit_app.py
```

Then open:

- API docs: http://localhost:8000/docs
- Dashboard: http://localhost:8501

WITH DOCKER (for deployment)

bash

```
docker-compose up --build
```

Same URLs work.

WHAT TO VERIFY

1. http://localhost:8000/health returns {"status": "healthy"}
2. http://localhost:8000/docs shows interactive API documentation
3. http://localhost:8000/parts returns list of 8 parts
4. http://localhost:8501 shows the Streamlit dashboard
5. Dashboard "Ask a question" page returns grounded answers

---

7. Testing Your API

USING THE BROWSER

Open http://localhost:8000/docs — this is Swagger UI, auto-generated by FastAPI. You can test every endpoint by clicking "Try it out."

USING CURL (command line)

```bash
# Health check
curl http://localhost:8000/health

# List parts
curl http://localhost:8000/parts

# Ask a question
curl -X POST http://localhost:8000/query \
  -H "Content-Type: application/json" \
  -d '{"question": "Which parts are at stockout risk?"}'

# Get metrics
curl http://localhost:8000/metrics
```

USING PYTHON

```python
import requests

response = requests.post(
    "http://localhost:8000/query",
    json={"question": "Which parts are at stockout risk?"}
)
print(response.json()["answer"])
```

---

8. Interview Preparation

QUESTION 1: "Describe your deployment architecture."

YOUR ANSWER:
"I deployed the system as two microservices: a FastAPI REST API serving

ML forecasts and RAG queries on port 8000, and a Streamlit dashboard for visual interaction on port 8501. Both are containerized with Docker and orchestrated with docker-compose. The API uses lazy loading for the RAG pipeline and ML models — they load on first request, keeping startup under 1 second. CORS middleware enables cross-origin communication between the dashboard and API."

QUESTION 2: "Why FastAPI over Flask?"

YOUR ANSWER:
"Three reasons. First, automatic API documentation via Swagger UI — every endpoint is documented and testable without writing docs manually. Second, request validation via Pydantic models — malformed requests get rejected with detailed error messages before hitting business logic. Third, async support — FastAPI handles concurrent requests efficiently, which matters when multiple users query the RAG pipeline simultaneously."

QUESTION 3: "How would you deploy this to production?"

YOUR ANSWER:
"I'd deploy to AWS using ECS (Elastic Container Service) or EKS (Kubernetes). The Docker image pushes to ECR (container registry), ECS pulls it and runs it behind an ALB (Application Load Balancer) for HTTPS and auto-scaling. The knowledge base would move from local ChromaDB to a managed vector database like Pinecone or AWS OpenSearch. ML models would be served via SageMaker for auto-scaling inference. CI/CD via GitHub Actions: push to main triggers tests, builds the Docker image, and deploys to staging automatically."

QUESTION 4: "What about security?"

YOUR ANSWER:
"Several layers. API keys for authentication — the /query endpoint would require a Bearer token. HTTPS via ALB termination. Environment variables for secrets (OpenAI key never in code). CORS restricted to specific origins in production (not allow_origins=['*']). Rate limiting to prevent abuse. Input sanitization on the /query endpoint to prevent prompt injection."

QUESTION 5: "Walk me through the full SDLC of this project."

YOUR ANSWER:
"Requirements: supply chain managers need demand forecasts accessible via natural language. Design: 4-layer architecture — ETL, ML, GenAI, deployment. Development: Phase 1 built the data pipeline with 36 tests,

Phase 2 built XGBoost and ARIMA with 21 features, Phase 3 built the RAG pipeline with ChromaDB and GPT-4, Phase 4 wrapped everything in FastAPI and Streamlit. Testing: 36 unit tests for ingestion, model evaluation metrics, RAG retrieval precision. Deployment: Dockerized with docker-compose, ready for AWS ECS. Monitoring: health endpoint, model metrics endpoint, MLflow tracking for experiment history."